Deployment Plan

1) Core runtime plan (keep-alive on a server)

Goal

A “hot running” instance where the model is already loaded in memory (so requests are fast).

Flow

1. User uploads an image from web UI
2. Server receives image and validates it
3. Server preprocesses the image with crop and optionally with lab normalize
4. Ensemble runs predictions on image using CPU to keep down cost
5. Server returns: ensemble “vote” and individual member predictions
6. Server logs: latency, model version, request metadata, errors, ip and MAC address

---

2) Hosting decision (App Runner vs Elastic Beanstalk)

Candidates I tested

- **AWS App Runner**
- **AWS Elastic Beanstalk (EB)**

What happened

- App Runner took many attempts and ultimately wasn’t viable for the Docker images. It always got stuck on the healthcheck stage. This was a several day ordeal.
- EB *did* work with a Hello World Streamlit test, making it the pragmatic choice for this project’s “get it live reliably” requirement.

Why EB fits this use case

- It’s friendlier to “bring your own container / web app” patterns
- It gives you a more traditional server-like surface area (logs, scaling knobs, env vars)
- It’s easier to reason about “one web process serving inference” without fighting platform constraint

3) UI plan (Streamlit now, Flask next)

Current

- Streamlit is excellent for rapid demos, but I've hit interface limitations
- Specifically: mobile usability issues with the Streamlit file uploader
- Slick, but fragile compared to competition

Next step

- Move the inference endpoint to **Flask** (or FastAPI) for a more controllable UI and cleaner API boundaries.
- Can keep Streamlit temporarily as a "demo front end" while Flask becomes the production-ish backend, or just replace Streamlit entirely.

---

4) Domain + disclaimers (Route 53 + Sad News)

Domain

- Route 53 points a real domain at the app (professional, shareable, demo-friendly).

Disclaimers to include prominently

This model will **not** work on "any ol' image." It will work best on images that match the training distribution:

- similar microscope/camera characteristics
- similar staining and lighting
- similar crop/scale (single centered cell, minimal clutter)
- not a medical device; not diagnostic; educational/demo use

---

5) "Care and nurturing" after deployment (monitoring + redeploy loop)

Deployment is the first day of the model's job. Then comes the ongoing life support:

Monitoring (minimum viable MLOps)

Track and alert on:

- **Latency** and timeouts
- **Error rate** (bad images, preprocessing failures, model exceptions)
- **Traffic** (requests/day, ip and mac address, routed through law enforcement or gov)
- **Confidence patterns** (average max softmax, entropy).
- **"Out-of-family" detection**: image size too small, too blurry, too dark, multiple cells, etc. For now, these mainly just get classified as "normoblast…"

Logging (what to store)

- model version / git commit hash
- request id
- Timestamp
- Source IP Address
- Destination IP Address
- Type of Service (DSCP/ECN)
- Mac address
- Traffic timestamps
- preprocessing stats (image dims, inferred brightness/contrast, blur score)
- inference time (ms)
- top-k predictions + probabilities
- Avoid storing raw images unless the user explicitly opts-in.

Retrain + redeploy plan

1. Collect failure cases (opt-in) and "low-confidence" images
2. Label/curate them into a new training set slice
3. Retrain and evaluate against:
    - original validation set
    - a "foreign microscope" challenge set (if available)

4. Export a versioned artifact: `model_vX_Y_Z.pt`
5. Deploy via a repeatable pipeline (build container → push → EB deploy)
6. Keep the previous version available for rollback
7. Archive a snapshot of all code from the deployment, tag with date (eventually delete)

6) How it fits into the larger ML pipeline

This pipeline becomes a loop instead of a line:

Data >

> Train >

> Evaluate >

> Package >

> Deploy >

> Monitor >

> Collect >

> Retrain >

> Redeploy

Key engineering connections:

- Preprocessing code must be shared between training and inference (single source of truth)
- Model artifacts must be versioned (weights + label map + preprocessing config)
- Metrics must be comparable across time (same eval setup)

---

7) Options awareness (what else exists, and why she/they didn't pick it)

I can show you understand the landscape even if she chose EB:

Open source / self-managed

- **Docker on a VPS** (simple, cheap, but ops are self-owned)
- **Kubernetes** (powerful, complex, "excellence" but not needed with EB)
- **TF Serving / TorchServe** (specialized serving layers, more moving parts)
- More attention to CI/CD (this can be ignored forever), should bootstrap with Kubernetes.

AWS

- **SageMaker Endpoints** (ML-native, expensive according to Quoara)
- **ECS/Fargate** (container orchestration, more control than EB)
- **Lambda** (Slow cold starts according to Stack Overflow)
- **API Gateway + Service** (I did not explore this option)

Other clouds

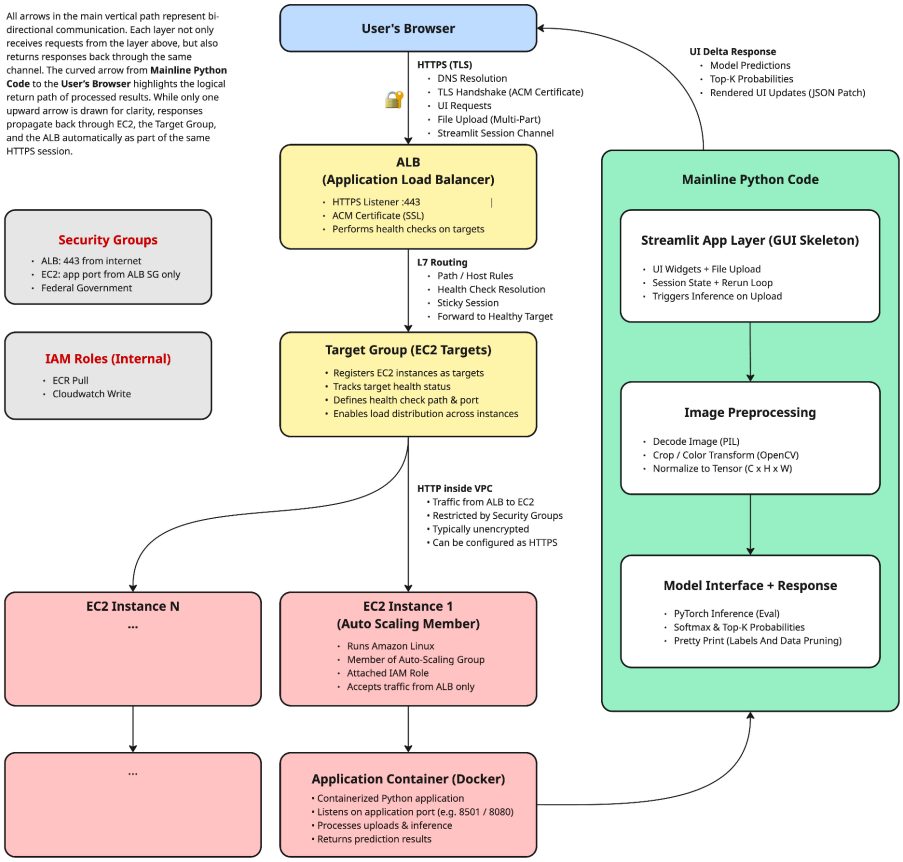
- **GCP Vertex AI**, **Azure ML**, **IBM Watson** equivalents: managed endpoints + monitoring. There's no time to try them all.

This justification is sufficient: ultimately EB was the platform that crossed the finish line. The website seems fine. It's setup to scale up. Probably take the bite from the bill.

---

8) Pseudocode + diagrams (rubric-friendly)

Architecture sketch



Flask-ish pseudocode for “model stays hot”

```
app = Flask(__name__)
MODELS = load_models("model1.pt", "model2.pt") # loaded once at startup
@app.post("/predict")
def predict():
  img = extract_image_from_request(request)
  x = preprocess(img)                # same as training
  probs = MODEL(x)                    # inference
  topk = decode_topk(probs, [“basophil”, …, “normoblast])
  log_request(latency_ms=..., topk=..., model_ver=...)
  triage_to_federal(source_ip, dest_ip, timestamp, mac_address, payload)
  return pretty_print(topk)
```

---

9) “Excellence” note (optional, but rubric-aware)

I can try this next, EB did not need Kubernetes. I got the site up, this was great.